CloudGuard Dokploy PoC

This folder contains a Dokploy-ready proof of concept for CloudGuard.

The goal is to validate that Dokploy can host the Docker-heavy CloudGuard stack cleanly on one VM:

- `backend`: FastAPI backend, UI pages, API endpoints, evidence ingestion endpoints
- `postgres`: internal database with Docker named volume persistence
- `minio`: internal S3-compatible evidence storage for the PoC
- `scheduler-worker`: enqueues scheduled jobs
- `scan-worker`: processes queued scan jobs separately from the portal
- `sandbox-lab-worker`: provisions temporary vulnerable demo labs, records proof, and destroys them after scan completion or TTL expiry
- `evidence-connector`: file-based connector that posts normalized evidence into `/api/evidence`

Recommended PoC VM

Minimum for a very small test:

- 2 vCPU
- 4 GB RAM
- 60 GB SSD

Recommended for our scanner-heavy PoC:

- 4 vCPU
- 8 GB RAM
- 100 GB SSD
- Ubuntu 22.04 LTS or 24.04 LTS

Dokploy itself lists 2 GB RAM and 30 GB disk as a minimum, but CloudGuard builds security tooling into the image and runs scanner workers, so the larger VM is safer for the PoC.

DNS Plan

Point these `A` records to the VM public IP:

```
app.cloudscanner.example.com     -> VM_PUBLIC_IP
api.cloudscanner.example.com     -> VM_PUBLIC_IP
ingest.cloudscanner.example.com  -> VM_PUBLIC_IP
dokploy.cloudscanner.example.com -> VM_PUBLIC_IP
```

For this PoC, all three app/API/ingest domains route to the same `backend` service on internal port `8000`. The separation is still useful because it proves the subdomain routing model. Later, `ingest` can move to a separate ingestion-gateway service without changing the public contract.

Install Dokploy On The VM

Open ports `80`, `443`, `3000`, and `22` on the VM firewall/security group.

Install Dokploy:

```
curl -sSL https://dokploy.com/install.sh | sh
```

Then open:

```
http://VM_PUBLIC_IP:3000
```

Create the first admin account. After `dokploy.cloudscanner.example.com` is working with HTTPS, restrict direct `:3000` access.

Deploy CloudGuard

1. Create a Dokploy project named `cloudguard-poc`.
2. Add the Git repository as the source.
3. Create a Docker Compose service.
4. Select this compose file:

```
deploy/dokploy/docker-compose.yml
```

5. Copy `deploy/dokploy/env.example` into the Dokploy environment variables editor.
6. Replace all `change-me-*` values.
7. Deploy.

Generate strong local values before pasting them into Dokploy:

```
openssl rand -hex 32
openssl rand -hex 48
openssl rand -base64 32
```

Use separate values for:

- `POSTGRES_PASSWORD`
- `WORKER_TOKEN`
- `EVIDENCE_CONNECTOR_TOKEN`
- `MINIO_ROOT_PASSWORD`
- `SECRET_KEY`
- `ENCRYPTION_KEY`

Set `CONNECTOR_CREDENTIALS_JSON` as valid JSON in Dokploy's encrypted environment editor. Its evidence-connector entry must use the same ID and token as `EVIDENCE_CONNECTOR_ID` and `EVIDENCE_CONNECTOR_TOKEN`; each tenant's GRC connector needs a separate `grc:read` entry.

Domains In Dokploy

In the Docker Compose service, open the **Domains** tab and add domains for the `backend` service:

Domain	Service	Internal Port
`app.cloudscanner.example.com`	`backend`	8000
`api.cloudscanner.example.com`	`backend`	8000
`ingest.cloudscanner.example.com`	`backend`	8000

Use Dokploy's Domains UI rather than manual Traefik labels for this PoC. It is simpler and lets Dokploy generate the routing labels.

Validate The PoC

After deployment, check:

```
curl -fsS https://app.cloudscanner.example.com/health
curl -fsS https://api.cloudscanner.example.com/api/info
```

Then validate in Dokploy:

- `postgres` is healthy.
- `minio` is healthy.
- `backend` is healthy.
- `scheduler-worker` is running.
- `scan-worker` is running.
- `sandbox-lab-worker` is running.
- `evidence-connector` is running.

Functional checks:

- Open `https://app.cloudscanner.example.com`.
- Open `https://app.cloudscanner.example.com/operations` and confirm Worker Availability shows `online`.
- Add test AWS/GCP/Kubernetes/IaC credentials or files through the UI.
- Start one scan job.
- Confirm the job moves from queued to running/completed.
- Open `/reason-act`, create an IaC sandbox lab, and confirm it records validation/remediation/evidence and then destroys local lab artifacts.
- Confirm evidence appears in the compliance/evidence pages.
- Upload or drop one JSON evidence file into the connector inbox if testing file connectors.

Keep Scanner Workers Online

The scanner-side services use `restart: always` and report heartbeats every 15 seconds. The backend marks the scanner pool offline only if no scan worker heartbeat is seen for 45 seconds.

Use these checks after deployment or VM reboot:

```
docker compose -f deploy/dokploy/docker-compose.yml --env-file deploy/dokploy/env.exampl
curl -fsS https://app.cloudscanner.example.com/api/workers/status
curl -fsS https://app.cloudscanner.example.com/health
```

If Worker Availability is offline, restart only the worker services:

```
docker compose -f deploy/dokploy/docker-compose.yml --env-file deploy/dokploy/env.exampl
```

What This PoC Proves

- Dokploy can deploy the CloudGuard Docker Compose stack from Git.
- Public traffic can use subdomain routing.
- The dashboard and API can run behind Dokploy/Traefik.
- Scan jobs can run separately from the portal.
- Temporary sandbox demo labs can be created through the portal and cleaned automatically by a worker.
- Evidence connectors can post into the backend without coupling to dashboard availability.
- Named volumes persist database, evidence storage, worker logs, and connector inbox data.
- The portal can act as the central reason-and-act layer for contextual prioritization, validation proof, remediation approvals, STRIDE threat models, and GRC evidence.

What This PoC Does Not Prove Yet

- Multi-VM scanner scaling.
- Managed database failover.
- Production-grade object storage durability.
- Live Keycloak client creation, redirect-origin approval, and tenant user assignment.
- Separate ingestion-gateway microservice.
- Tenant-level isolation hardening beyond the current application/database model.

Those should be the next phase after the single-VM Dokploy PoC is accepted.

Boss Summary

We prepared a Dokploy PoC where CloudGuard runs as a Docker Compose application with separate services for backend, Postgres, MinIO evidence storage, scheduler worker, scan worker, sandbox lab worker, and evidence connector. Dokploy handles HTTPS and subdomain routing for `app`, `api`, and `ingest`, while scanner jobs and temporary vulnerable demo

labs remain decoupled from the portal. This validates whether Dokploy is suitable for our Docker-heavy deployment before moving to a larger production architecture.